RELATÓRIO DE AUDITORIA DE SEGURANÇA

Segundo Cérebro AI (Second Brain)

Data: 17/10/2026 **Total:** 8 achados (1 alta, 1 média, 1 baixa, 5 informativa)

Escopo auditado: Repositório inteiro no commit 90e2381 (branch main): frontend Vite/JS (src/, public/, index.html), Edge Functions Deno (supabase/functions), SQL e RLS (supabase/auth_rls.sql e 4 migrations), workflow n8n, serviço media-fetcher (Python), Dockerfile, nginx.conf, security-headers.conf, GitHub Actions, testes e documentação. Histórico git completo (20 commits) para a busca de segredos.

Nota metodológica

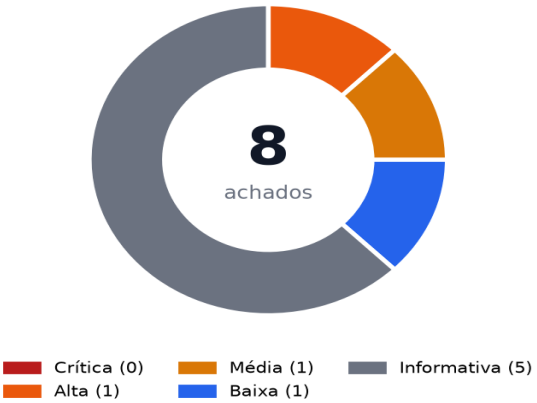
Esta auditoria mapeia cinco categorias de falha para a stack real do projeto, mais uma sexta para achados verificados fora delas. Só são reportados achados confirmados no código; o que é esperado por design aparece como informativo.

Stack detectada	SPA/PWA em Vite com JavaScript puro (sem framework); backend Supabase (Postgres, pgvector, Auth e Edge Functions Deno); pipeline de ingestão em n8n; serviço media-fetcher (Python, yt-dlp, ffmpeg); deploy por Dockerfile (nginx) via Coolify, disparado pelo GitHub Actions.
1. Banco sem tranca	Usuário único: o isolamento é por RLS com public.is_owner() (auth.uid() igual ao dono gravado em brain_owner), não por coluna de tenant. Foram verificadas todas as tabelas, políticas, funções RPC e grants, e também o que a tranca aceita como prova de identidade (fator único ou MFA).
2. Permissão no navegador	Não existem papéis (isAdmin, canEdit, role) no frontend; o único gate do cliente é ter sessão (main.js). Foi conferido se o backend impõe a regra do dono em toda rota: RLS, requireOwner nas Edge Functions e os nós Exigir Token e Verificar Dono no n8n.
3. IDOR	Sem multiusuário, IDOR significa acessar objeto por ID sem prova de posse. Foram percorridos todos os handlers que recebem ID: brain-process (item_id), as 8 funções de supabaseClient.js que recebem ID e as RPCs por ID (merge, duplicatas, exemplos, claim, replace).
4. Chaves expostas	Busca por padrões de segredo (chaves Google, JWT, tokens GitHub/Slack, chaves privadas, service_role, defaults ${VAR:-valor}) na árvore e em todo o histórico git, mais leitura manual de CI, Dockerfiles, workflow n8n, READMEs, skill versionada e bundle (config.js).
5. Inputs sem tratamento	Frontend: todas as 50 ocorrências de innerHTML, insertAdjacentHTML e href em 16 arquivos de src/, o renderizador de Markdown, o service worker e a CSP. Backend: filtros PostgREST montados por string e texto exportado. Nenhum exploit foi executado contra ambiente real.
6. Adicionais	Achados verificados fora das cinco categorias, principalmente SSRF: todo ponto em que uma URL vinda de conteúdo capturado vira requisição (Edge Functions, n8n e media-fetcher).

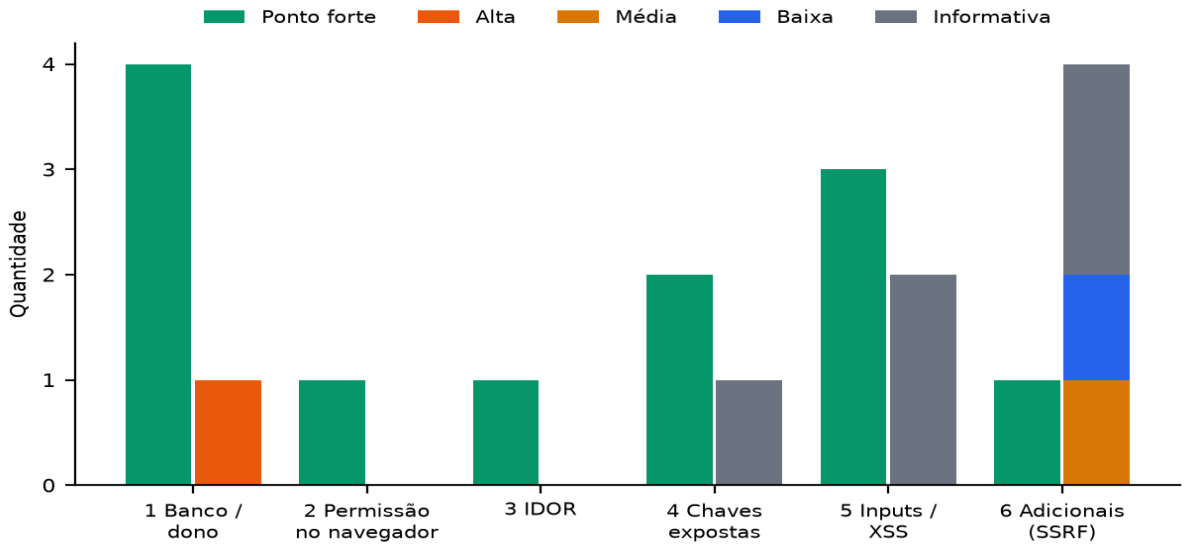
Resumo executivo

Não há falhas críticas, e as cinco categorias pedidas estão bem cobertas: RLS restrito ao dono em todas as tabelas, funções com security invoker, nenhum segredo real no código ou no histórico, e nenhum XSS (tudo passa por escapeHtml, com uma CSP estrita por trás). Os 183 testes passam, incluindo 9 de RLS. Os dois pontos que pedem ação são: **o MFA indicado no setup não é exigido em lugar nenhum** (a senha sozinha dá acesso total) e **o yt-dlp do media-fetcher contorna a proteção de SSRF**, alcançando a rede interna do Docker.

Severidade	Quantidade
CRÍTICA	0
ALTA	1
MÉDIA	1
BAIXA	1
INFORMATIVA	5
PONTO FORTE	12



Achados e pontos fortes por categoria



Pontos fortes e pontos fracos

Pontos fortes (com evidência)

Cat.	Ponto forte	Evidência
1	**RLS ativo em todas as tabelas, com política única do dono**	base_schema_rls.sql:57-69 (brain_items, brain_chunks, remove políticas antigas em 44-55); knowledge_atoms.sql:168-174 (brain_atoms); duplicates.sql:83-89 (brain_duplicate_decisions).
1	**Tabela do dono fechada e is_owner() blindada**	auth_rls.sql:13-15 (RLS + revoke all de public/anon/authenticated); :28-36 (security definer com search_path fixo); :21-23 (recusa o UUID de zeros).
1	**Todas as RPCs com security invoker e sem execução para anon**	knowledge_atoms.sql:509-519; duplicates.sql:238-246; practical_examples.sql:26-27; base_schema_rls.sql:73-85 força invoker na legada match_brain_chunks.
1	**RLS coberto por testes automatizados**	tests/db/rls.test.mjs:29-141 (9 testes: dono, não-dono, anon, uid nulo, troca de dono); npm test: 183 de 183 passando em 17/10/2026.
2	**Sem gates de papel no cliente; o backend impõe a regra do dono em toda rota**	main.js:518-537 só alterna login/app pela sessão; runtime.ts:51-67 (requireOwner via owner_check com o token do chamador, sem service role); config.toml:5-9 (verify_jwt); n8n: nós Exigir Token e Verificar Dono antes de qualquer processamento.
3	**Acesso por ID sempre sob RLS**	brain-process/index.ts:10 e :43 (UUID validado; cliente com o token do usuário); supabaseClient.js (8 funções por ID, todas com a anon key + sessão); duplicates.sql:186-236 (merge com for update sob RLS).
4	**Nenhum segredo real no código nem no histórico git**	git grep e git log -p --all -G (20 commits) sem chaves Google, JWT service_role, tokens ou chaves privadas; .gitignore:25-35 e .dockerignore cobrem .env, *.pem, *.key e coolify_private_key.txt.
4	**Segredos de runtime fora do código, com validação**	deploy.yml:16-35 (secrets do GitHub, HTTPS obrigatório, permissions: contents: read); server.py:395-396 (não sobe sem FETCHER_API_KEY) e :373 (hmac.compare_digest); credenciais do n8n por referência; GEMINI_API_KEY em secrets do Supabase (runtime.ts:69-77).
5	**Escape consistente em todo HTML dinâmico**	escapeHtml em todas as 50 ocorrências de innerHTML/insertAdjacentHTML (16 arquivos); markdown.js escapa antes de formatar e só gera links http(s); href checado por regex em ItemDetail.js:174 e KnowledgeView.js:9.
5	**CSP estrita e cabeçalhos de segurança**	security-headers.conf:3 (script-src 'self', object-src 'none', frame-ancestors 'none', base-uri e form-action 'self'); :4-8 (nosniff, DENY, HSTS, Referrer e Permissions-Policy); theme-init.js externo para dispensar unsafe-inline.
5	**Filtros PostgREST e captura externa tratados**	postgrest.js:2 remove , ( ) % * \ " dos termos (usado em supabaseClient.js:50 e :229); CaptureModal.js:22-24 e :188 avisam origem externa e exigem confirmação manual.
6	**Guardas de SSRF sólidos no caminho principal**	ssrf.ts:32-117 (IPv4/IPv6 privados, NAT64, 6to4, A+AAAA obrigatórios, portas 80/443) e :137-175 (revalida cada redirect, timeout e limite de bytes); server.py:81-128 (IP fixado contra rebinding no download direto); media-fetcher roda sem root, com yt-dlp pinado (Dockerfile:8-20).

Pontos fracos (resumo)

ID	Sev.	Ponto fraco
1.1	ALTA	MFA recomendado no setup, mas nunca exigido: só a senha já dá acesso total
6.1	MÉDIA	SSRF no media-fetcher: o yt-dlp refaz o acesso sem as checagens do preflight
6.2	BAIXA	Janela de DNS rebinding entre a checagem por DoH e o fetch (Edge Functions e n8n)
6.3	INFORMATIVA	Mensagens de erro do banco devolvidas ao cliente
6.4	INFORMATIVA	Código morto chama RPC legada cujo corpo não está versionado
5.1	INFORMATIVA	Share-target GET converte ?url= externo em shared_*, contrariando o comentário
4.1	INFORMATIVA	Anon key do Supabase no bundle (esperado)
5.2	INFORMATIVA	Token de sessão em localStorage (padrão do supabase-js)

Tabela de achados por categoria

1. Banco sem tranca (isolamento de dono)

Severidade	Arquivo:linha	Descrição

ALTA	supabase/auth_rls.sql:5
	supabase/auth_rls.sql:28-36
	src/components/LoginScreen.js:43
	src/shared/services/supabaseClient.js:10-14
		[1.1] MFA recomendado no setup, mas nunca exigido: só a senha já dá acesso total
		O passo 4 do setup manda ativar o TOTP e cadastrar o autenticador do dono, mas nenhuma camada exige o segundo fator. O login chama apenas signInWithPassword, sem mfa.challenge nem mfa.verify; e is_owner(), que é a tranca de todas as políticas RLS, das Edge Functions (owner_check) e do n8n, só compara auth.uid(). No Supabase, uma sessão aal1 (só senha) continua válida mesmo com fator TOTP cadastrado.
		Evidência: `auth_rls.sql:5 -- 4) Em Authentication > Multi-Factor, ative o TOTP e cadastre o autenticador do dono.`
		`auth_rls.sql:35 select exists (select 1 from public.brain_owner where user_id = auth.uid())`
		`LoginScreen.js:43 await signInWithPassword(emailInput.value.trim(), passwordInput.value);`
		`grep -ri 'mfa|aal|challenge' src supabase -> nenhuma ocorrência fora do comentário do setup`
		Condições de explorabilidade: Atacante com a senha do dono (phishing, vazamento ou reuso de senha). Não precisa de nenhuma configuração insegura: é o comportamento padrão com o código atual.
		Impacto: Leitura, alteração e exclusão de todo o acervo, além do disparo do pipeline (custo de Gemini). O dono acredita estar protegido por MFA e não está.
		Issue: Issue 1

2. Permissão definida no navegador

Sem achados. Verificado em todos os pontos aplicáveis: Sem gates de papel no cliente; o backend impõe a regra do dono em toda rota.

3. IDOR

Sem achados. Verificado em todos os pontos aplicáveis: Acesso por ID sempre sob RLS.

4. Chaves expostas (hardcode)

Severidade	Arquivo:linha	Descrição
INFORMATIVA	src/shared/constants/config.js:5-6	**[4.1] Anon key do Supabase no bundle (esperado)** A anon key (role anon) é publicável por design; a proteção real é o RLS, que nega tudo ao anon (verificado em tests/db/rls.test.mjs:42-58). Registrado para mostrar que foi avaliado. **Evidência:** `config.js:6 anonKey: 'eyJhbGciOiJIUzI1NiIs...' (payload: role = anon)` **Condições de explorabilidade:** Nenhuma: a chave não dá acesso a dados. **Impacto:** Nenhum enquanto o RLS continuar ativo em todas as tabelas. **Issue:** não acionável (esperado por design)

5. Inputs sem tratamento (XSS)

Severidade	Arquivo:linha	Descrição
INFORMATIVA	public/sw.js:67-75 src/shared/utils/sharedParams.js:1-2 public/manifest.json:26	**[5.1] Share-target GET converte ?url= externo em shared_*, contrariando o comentário** sharedParams.js diz que só os shared_* gerados pelo service worker valem, mas o próprio worker transforma qualquer /share-target?url=... em shared_url. O manifest só declara POST (manifest.json:26), então esse ramo GET só é alcançado por links externos. Como qualquer pessoa também pode montar /?shared_url=... direto, a defesa real é o aviso de origem externa com confirmação manual (CaptureModal.js:22-24 e :188), que funciona. **Evidência:** `sw.js:70 Response.redirect('/?shared_url=' + encodeURIComponent(url.searchParams.get('url') || '') + ...` **Condições de explorabilidade:** O dono precisa abrir o link e confirmar o salvamento apesar do aviso. **Impacto:** Pré-preenchimento de captura por terceiros, já mitigado pelo aviso. **Issue:** Issue 4
INFORMATIVA	src/shared/services/supabaseClient.js:5-8 src/shared/services/captureService.js:125	**[5.2] Token de sessão em localStorage (padrão do supabase-js)** O supabase-js guarda a sessão em localStorage, legível por qualquer script da origem, e o token é repassado ao n8n. Não há XSS conhecido e a CSP bloqueia script inline e de terceiros, então isso é defesa em profundidade, não falha. **Evidência:** `captureService.js:125 Authorization: \`Bearer ${session.access_token}\`` **Condições de explorabilidade:** Exigiria um XSS, que não foi encontrado. **Impacto:** Sequestro de sessão caso surja um XSS no futuro. **Issue:** não acionável (esperado por design)

6. Achados adicionais (SSRF e higiene)

Severidade	Arquivo:linha	Descrição

MÉDIA	media-fetcher/server.py:331-336
	media-fetcher/server.py:216-231
	n8n/second_brain_workflow.json (nó Preparar Conteúdo, jsCode:208 e :277)
		[6.1] SSRF no media-fetcher: o yt-dlp refaz o acesso sem as checagens do preflight
		fetch_media() valida a URL com preflight() (DNS resolvido, IP fixado, redirects revalidados) e em seguida entrega a mesma URL ao yt-dlp, que resolve DNS de novo, segue redirects sozinho e, no extrator genérico, baixa URLs embutidas na página (video src, og:video, manifestos HLS/DASH) para qualquer host e porta. O n8n envia ao fetcher links de redes sociais e também qualquer link web que responda com vídeo ou áudio acima de 14 MB (fetchUrl = target).
		Evidência: `server.py:332 preflight(url)`
		`server.py:336 download_with_ytdlp(url, workdir) # yt-dlp não usa resolve_public/IP fixado`
		`server.py:226 cmd.append(url) # sem restrição de rede, porta ou redirect`
		`n8n jsCode:277 if (out.needsFetcher) out.fetchUrl = target;`
		Condições de explorabilidade: O dono captura um link controlado pelo atacante (ex.: recebido por mensagem). O servidor do atacante responde com mídia às duas primeiras requisições (n8n e preflight) e, para o yt-dlp, com redirect ou página apontando para um endereço interno; DNS rebinding também funciona.
		Impacto: Requisições GET cegas, a partir do container, à rede interna do Docker (n8n em :5678, outros serviços do Coolify) e ao serviço de metadados da nuvem (169.254.169.254), em qualquer porta. Mídia obtida pode acabar na nota do dono.
		Issue: Issue 2

BAIXA	supabase/functions/_shared/ssrf.ts:80-88
	supabase/functions/_shared/ssrf.ts:144-148
	n8n/second_brain_workflow.json (nó Preparar Conteúdo, jsCode:138-141 e :149-150)
		[6.2] Janela de DNS rebinding entre a checagem por DoH e o fetch (Edge Functions e n8n)
		assertPublicUrl resolve o host via DNS-over-HTTPS e confere os IPs, mas o fetch seguinte resolve o nome de novo pelo resolvedor do sistema. Um domínio com TTL zero pode responder IP público na checagem e IP interno no acesso. As URLs varridas pela Edge Function vêm do conteúdo capturado (átomos de link), portanto o atacante as controla.
		Evidência: `ssrf.ts:83 addresses = await resolve(host); // checagem via DoH`
		`ssrf.ts:148 const response = await fetchImpl(current, { redirect: 'manual', ... }); // nova resolução`
		`n8n jsCode:139-150 resolveHost(host) ... this_helpers.httpRequest({ url: current, ... })`
		Condições de explorabilidade: Domínio do atacante com DNS próprio e TTL baixo, citado em conteúdo que o dono captura. O alcance fica limitado às portas 80 e 443 (assertAllowedPort); na Edge Function, a rede é a da infraestrutura do Supabase, não a do Coolify.
		Impacto: GET cego a serviços internos nas portas 80/443 (no n8n, os containers da mesma rede Docker).
		Issue: Issue 3

INFORMATIVA	supabase/functions/brain-process/index.ts:41 supabase/functions/brain-process/index.ts:56 supabase/functions/_shared/runtime.ts:90-111	**[6.3] Mensagens de erro do banco devolvidas ao cliente** Erros do PostgREST (error.message) são concatenados e devolvidos na resposta e nos warnings do brain-process. Hoje só o dono os recebe, então o impacto é mínimo, mas detalhes de esquema não deveriam sair do servidor. **Evidência:** `index.ts:41 return json(req, { error: \`Falha ao listar pendentes: ${error.message}\` }, 500);` `index.ts:56 warnings: [error instanceof Error ? error.message : String(error)]` **Condições de explorabilidade:** Só o dono autenticado vê as respostas. **Impacto:** Vazamento de detalhes internos (nomes de colunas e funções) se a função passar a ter outros usuários. **Issue:** Issue 4

Severidade	Arquivo:linha	Descrição
INFORMATIVA	src/shared/services/supabaseClient.js:104-116 supabase/migrations/20261017110000_base_schema_rls.sql:71-85	**[6.4] Código morto chama RPC legada cujo corpo não está versionado** searchBrainSemantics() chama match_brain_chunks, função que existe só em produção: a migration apenas a converte para security invoker e fecha para anon, sem versionar o corpo. Nenhum arquivo chama searchBrainSemantics. **Evidência:** `supabaseClient.js:105 const { data, error } = await supabase.rpc('match_brain_chunks', {` `grep -rn searchBrainSemantics src -> só a definição` **Condições de explorabilidade:** Nenhuma hoje (a migration já força security invoker). **Impacto:** Superfície sem revisão possível: o corpo da função não pode ser auditado pelo repositório. **Issue:** Issue 4

Recomendações priorizadas

P1 — Imediato

Issue	Sev.	Ação
#1	ALTA	Implementar o desafio TOTP no login e exigir aal2 em is_owner(), nessa ordem, para não trancar o dono fora do app.

P2 — Próximos dias

Issue	Sev.	Ação
#2	MÉDIA	Fechar o egress do media-fetcher para a rede interna e impedir que o yt-dlp siga destinos não validados.

P3 — Próximas semanas

Issue	Sev.	Ação
#3	BAIXA	Bloquear redes privadas na saída do container do n8n; aceitar o risco residual na Edge Function ou passar o fetch por um proxy com IP fixado.

P4 — Backlog

Issue	Sev.	Ação
#4	INFORMATIVA	Remover a RPC legada e o código morto, padronizar mensagens de erro e alinhar o share-target GET.

ISSUES PARA O GITHUB

4 issues cobrem os 6 achados acionáveis (os três informativos de higiene foram unidos na issue 4). Copie o texto entre os marcadores de cada issue.

Issue 1 — [Segurança] Exigir MFA (aal2) no RLS e no login: hoje a senha sozinha dá acesso total

```
--- ISSUE 1 ---
Título: [Segurança] Exigir MFA (aal2) no RLS e no login: hoje a senha sozinha dá acesso total
Labels sugeridas: security, severity:alta

## Problema
O setup (supabase/auth_rls.sql, passo 4) manda ativar o TOTP, mas nada exige o segundo fator. O
login usa só signInWithPassword, sem mfa.challenge/mfa.verify, e public.is_owner(), base de todas as
políticas RLS, do owner_check das Edge Functions e do n8n, compara apenas auth.uid(). No Supabase,
uma sessão aal1 (só senha) continua válida mesmo com fator cadastrado. Quem obtiver a senha do dono
tem acesso total.

## Evidência
supabase/auth_rls.sql:5
    -- 4) Em Authentication > Multi-Factor, ative o TOTP e cadastre o autenticador do dono.
supabase/auth_rls.sql:35
    select exists (select 1 from public.brain_owner where user_id = auth.uid())
src/components/LoginScreen.js:43
    await signInWithPassword(emailInput.value.trim(), passwordInput.value);
Nenhuma ocorrência de mfa, aal ou challenge em src/ ou supabase/functions/.

## Impacto
Leitura, alteração e exclusão de todo o acervo e uso do pipeline (custo de Gemini) com uma senha
vazada ou obtida por phishing. O MFA sugerido no setup dá falsa sensação de segurança.

## Sugestão de correção
- Frontend primeiro: depois de signInWithPassword, chamar
  supabase.auth.mfa.getAuthenticatorAssuranceLevel(); se nextLevel = aal2 e currentLevel = aal1,
  mostrar um campo de código e chamar mfa.challenge + mfa.verify antes de exibir o app.
- Depois, no banco, exigir aal2 dentro de is_owner() (cobre RLS, Edge Functions e n8n de uma vez):
```sql
create or replace function public.is_owner() returns boolean
language sql stable security definer set search_path = public, pg_temp as $$
  select exists (select 1 from public.brain_owner where user_id = auth.uid())
    and coalesce(auth.jwt() ->> 'aal', 'aal1') = 'aal2'
$$;
```
- Versionar a mudança numa migration nova e atualizar o README com a ordem: cadastrar fator,
  publicar o frontend e só então aplicar a migration.

Critérios de aceite
- [ ] Login com senha correta e sem código TOTP não exibe dados e não consegue ler brain_items pela
  API.
- [ ] Com o código TOTP correto, o app funciona normalmente, incluindo a captura pelo n8n e o brain-
  ask.
- [ ] Teste em tests/db/rls.test.mjs: o dono com aal1 é negado em select/insert/update/delete; com
  aal2 tem acesso.
- [ ] is_owner() continua security definer com search_path fixo (o teste existente segue passando).

--- FIM ISSUE 1 ---
```

Issue 2 — [Segurança] SSRF no media-fetcher: yt-dlp refaz o acesso sem as checagens do preflight

```
--- ISSUE 2 ---
Título: [Segurança] SSRF no media-fetcher: yt-dlp refaz o acesso sem as checagens do preflight
Labels sugeridas: security, severity:media

## Problema
fetch_media() valida a URL com preflight() e depois a entrega ao yt-dlp, que resolve DNS de novo,
segue redirects e baixa URLs embutidas na página (video src, og:video, manifestos HLS/DASH) sem
nenhuma restrição de IP ou porta. O n8n manda ao fetcher links de redes sociais e qualquer link web
que responda com vídeo ou áudio maior que 14 MB.

## Evidência
media-fetcher/server.py:331-336
    def fetch_media(url):
        preflight(url)
        ...
            download_with_ytdlp(url, workdir)
media-fetcher/server.py:226
    cmd.append(url)
n8n/second_brain_workflow.json, nó Preparar Conteúdo (jsCode:277)
    if (out.needsFetcher) out.fetchUrl = target;

## Impacto
Com um link malicioso capturado pelo dono, o atacante faz o container emitir GETs cegos para a rede
interna do Docker (n8n em :5678, serviços do Coolify) e para o serviço de metadados da nuvem
(169.254.169.254), em qualquer porta.

## Sugestão de correção
- Fechar o egress do container do media-fetcher para faixas privadas e link-local (rede Docker
  dedicada, sem rota para as outras redes do Coolify, mais regras de firewall bloqueando 10/8,
  172.16/12, 192.168/16 e 169.254/16). Essa é a correção robusta.
- No código, passar ao yt-dlp a URL final já validada pelo preflight e usar --no-playlist quando o
  host não for de rede social.
- Para links web genéricos, preferir download_direct (IP fixado) e usar o yt-dlp só para os hosts da
  lista SOCIAL do n8n.

## Critérios de aceite
- [ ] Um servidor de teste que responde mídia ao preflight e 302 para http://169.254.169.254/ ao yt-
  dlp resulta em erro, sem requisição à faixa interna (confirmado por log de rede ou tcpdump).
- [ ] De dentro do container, curl para http://n8n:5678 e http://169.254.169.254 falha.
- [ ] Teste em media-fetcher/test_server.py para o caminho genérico (sem rede social) que não use
  yt-dlp com URL não validada.
- [ ] Instagram, TikTok e YouTube continuam funcionando.

--- FIM ISSUE 2 ---
```

Issue 3 — [Segurança] Janela de DNS rebinding no crawler (Edge Functions) e no n8n

```
--- ISSUE 3 ---
Título: [Segurança] Janela de DNS rebinding no crawler (Edge Functions) e no n8n
Labels sugeridas: security, severity:baixa

## Problema
assertPublicUrl (supabase/functions/_shared/ssrf.ts) e assertPublicUrl do nó Preparar Conteúdo
resolvem o host por DNS-over-HTTPS e validam os IPs, mas o fetch seguinte resolve o nome de novo. Um
domínio com TTL zero responde IP público na checagem e IP interno no acesso. As URLs varridas vêm do
conteúdo capturado.

## Evidência
supabase/functions/_shared/ssrf.ts:83
    addresses = await resolve(host);
supabase/functions/_shared/ssrf.ts:148
    const response = await fetchImpl(current, { redirect: 'manual', signal: controller.signal, ...
    });
n8n/second_brain_workflow.json, nó Preparar Conteúdo (jsCode:139 e :150)

## Impacto
GET cego a serviços internos nas portas 80/443. No n8n, alcança os containers da mesma rede Docker.

## Sugestão de correção
- n8n: bloquear redes privadas na saída do container (mesma regra de firewall da issue do media-
  fetcher). A API this.helpers.httpRequest não permite fixar o IP.
- Edge Function: documentar o risco residual (roda na infraestrutura do Supabase, sem a rede do
  Coolify) ou passar a varredura pelo media-fetcher, que já fixa o IP validado (server.py:81-128).

## Critérios de aceite
- [ ] De dentro do container do n8n, requisições a 10/8, 172.16/12, 192.168/16 e 169.254/16 falham.
- [ ] Decisão para a Edge Function registrada em supabase/README.md (risco aceito ou proxy
  implementado).

--- FIM ISSUE 3 ---
```

Issue 4 — [Segurança] Higiene: RPC legada não versionada, erros do banco na resposta e share-target GET

```
--- ISSUE 4 ---
Título: [Segurança] Higiene: RPC legada não versionada, erros do banco na resposta e share-target
GET
Labels sugeridas: security, severity:informativa

## Problema
Três ajustes pequenos agrupados: (a) searchBrainSemantics() é código morto e chama
match_brain_chunks, cujo corpo não está no repositório; (b) brain-process devolve error.message do
PostgREST ao cliente; (c) o share-target GET do service worker transforma qualquer ?url= externo em
shared_url, contrariando o comentário de sharedParams.js (o aviso de origem externa continua
mitigando).

## Evidência
src/shared/services/supabaseClient.js:104-116  (rpc('match_brain_chunks'), sem chamadores)
supabase/functions/brain-process/index.ts:41
    return json(req, { error: `Falha ao listar pendentes: ${error.message}` }, 500);
public/sw.js:70
    Response.redirect('/?shared_url=' + encodeURIComponent(url.searchParams.get('url') || '') + ...

## Impacto
Baixo: superfície que não pode ser revisada pelo repositório e vazamento de detalhes internos caso
surjam outros usuários.

## Sugestão de correção
- Remover searchBrainSemantics e, se nada mais usar, dropar match_brain_chunks numa migration (ou
  versionar o corpo dela).
- Em brain-process, registrar o erro com console.error e devolver mensagem genérica.
- Corrigir o comentário de sharedParams.js para refletir que a defesa é o aviso + confirmação, e
  remover o ramo GET de /share-target (o manifest só declara POST).

## Critérios de aceite
- [ ] grep por match_brain_chunks em src/ sem resultados; função removida ou versionada.
- [ ] Respostas 500 do brain-process não contêm mensagens do PostgREST.
- [ ] Comentário de sharedParams.js e comportamento do sw.js consistentes.

--- FIM ISSUE 4 ---
```